Container Technology

Docker & Kubernetes Administration & Security

Day 3

Kubernetes Security · Backup Strategies · Monitoring

Day 3 — Agenda

Part 1 — Kubernetes Administration & Security

- RBAC: Roles, ClusterRoles, Bindings, ServiceAccounts
- Admission Controllers: concept, mutating vs. validating
- Pod Security Admission (PSA) — levels and modes
- OPA/Gatekeeper: policy management with Rego
- Network policies and cluster segmentation
- Cluster hardening: CIS benchmarks, API server flags, etcd

Part 2 — Backup Strategies

- Why backup Kubernetes clusters
- What to back up — resources and volumes
- Velero: architecture, schedules, backup hooks
- Hands-on: namespace backup and restore

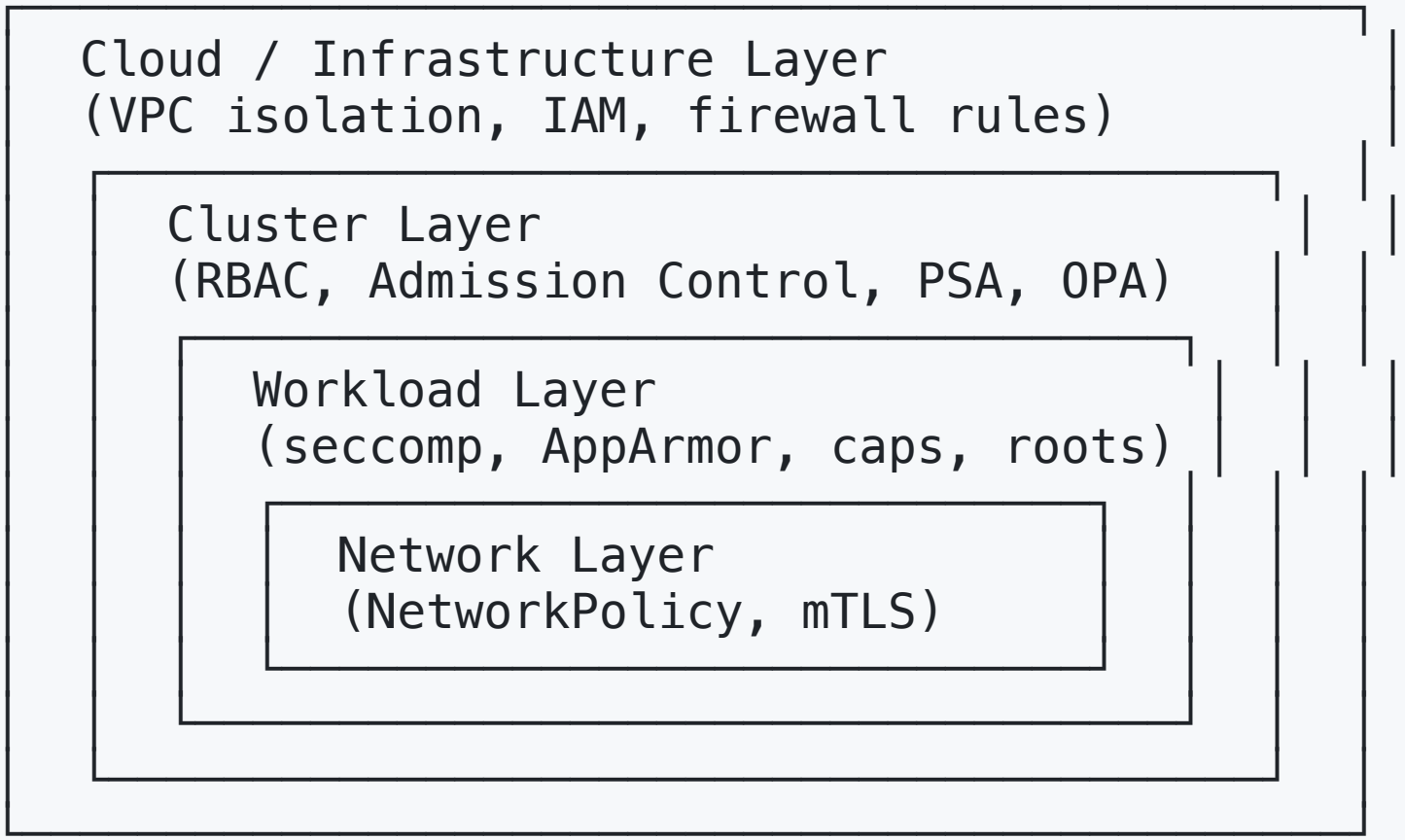
Part 3 — Monitoring

- Observability: metrics, logs, traces
- Prometheus: architecture, scraping, PromQL
- Alertmanager: routing and notifications
- Grafana: dashboards and visualisation
- kube-state-metrics and node-exporter
- Hands-on: deploy Prometheus & Grafana via Helm

Part 1 — Kubernetes Administration & Security

The Kubernetes Security Model

Kubernetes security is a **defence-in-depth** problem. No single control is sufficient.



RBAC — Role-Based Access Control

RBAC is Kubernetes's **authorisation** system. It controls who can do what to which resources.

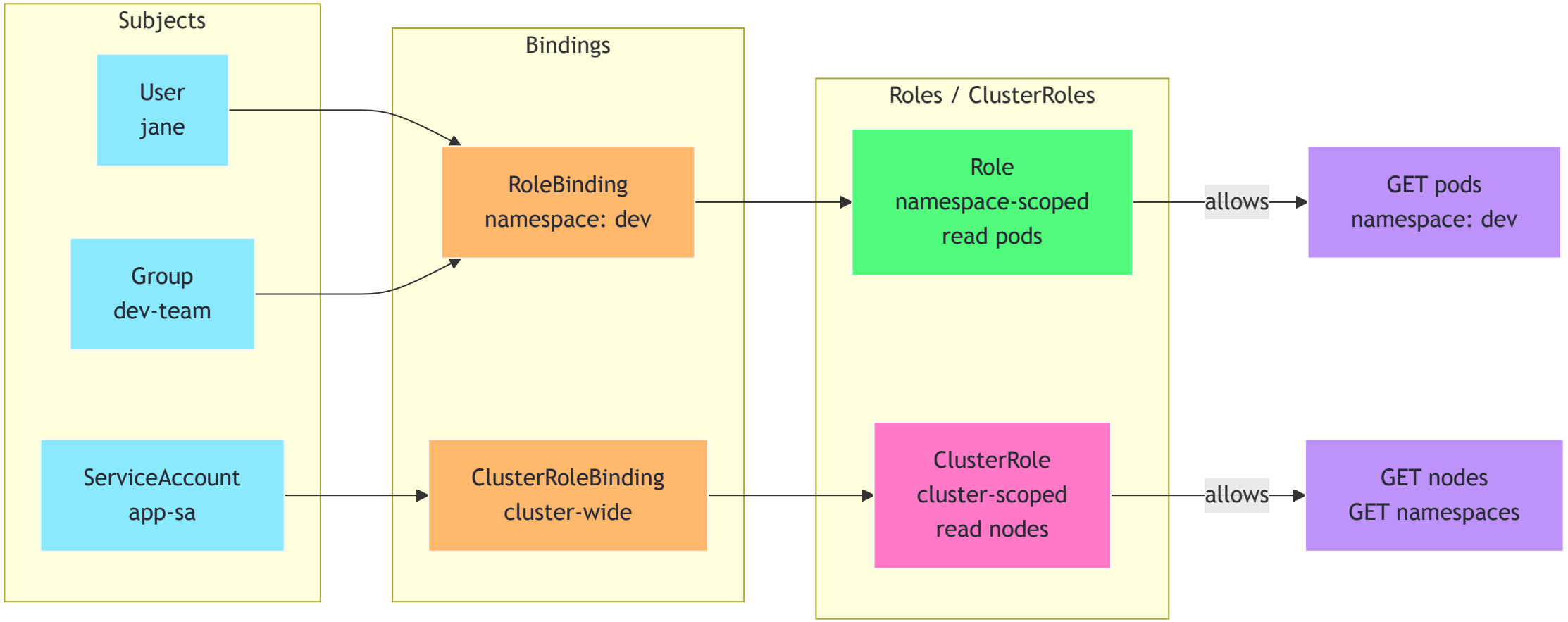
Four RBAC objects:

Object	Scope	Purpose
`Role`	Namespace	Grants permissions within a single namespace
`ClusterRole`	Cluster	Grants permissions across all namespaces or non-namespaced resources
`RoleBinding`	Namespace	Attaches a Role (or ClusterRole) to a subject within a namespace
`ClusterRoleBinding`	Cluster	Attaches a ClusterRole to a subject cluster-wide

Three subject types:

- `User` — human identity (authenticated via OIDC, certificate, token)
- `Group` — set of users
- `ServiceAccount` — identity for pods (applications)

RBAC Model


Subjects
User
jane
Group
dev-team
ServiceAccount
app-sa
Bindings
RoleBinding
namespace: dev
ClusterRoleBinding
cluster-wide
Roles / ClusterRoles
Role
namespace-scoped
read pods
ClusterRole
cluster-scoped
read nodes
allows
GET pods
namespace: dev
allows
GET nodes
GET namespaces

Writing RBAC Policies

```yaml
# Role — namespace-scoped, read-only on pods and logs
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: production
rules:
- apiGroups: [""]           # "" = core API group
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list"]

---
# RoleBinding — bind the role to a user and a group
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: production
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
- kind: Group
  name: dev-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

ServiceAccounts and RBAC

Every pod runs with a ServiceAccount. By default, the `default` ServiceAccount in each namespace has minimal permissions — but you should create dedicated ServiceAccounts for each application.

```
# Create a dedicated ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
automountServiceAccountToken: false   # opt-in, not opt-out
```

```yaml
# Grant the ServiceAccount specific permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: app-configmap-reader
  namespace: production
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  resourceNames: ["app-config"]   # named resource — tightest scoping
  verbs: ["get"]
```

> **Security:** Mount the ServiceAccount token only if the app actually calls the Kubernetes API. Most apps don't need it. Set `automountServiceAccountToken: false` by default.

Auditing RBAC — Common Tools

```
# Who can do what? (built-in)
kubectl auth can-i list pods --as jane --namespace production
kubectl auth can-i '*' '*' --as system:serviceaccount:production:app-sa

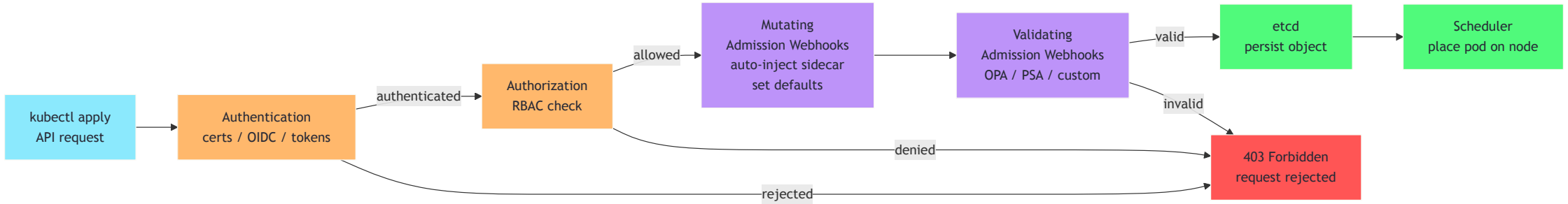
# What can a subject do? (requires auth-reconcile or rbac-lookup)
kubectl rbac-lookup jane -o wide

# Find all bindings for a user
kubectl get rolebindings,clusterrolebindings -A \
  -o json | jq '.items[] | select(.subjects[]?.name == "jane")'

# Detect overly broad permissions (e.g., wildcard verbs)
kubectl get clusterroles -o json | \
  jq '.items[] | select(.rules[]?.verbs[] == "*") | .metadata.name'
```

Admission Controllers

Admission controllers intercept API server requests **after authentication and authorisation** but **before persistence to etcd**.



Mutating vs. Validating Webhooks

Phase	Type	Examples
Mutating	Runs first, can modify the object	Inject sidecar, set default values, add labels
Validating	Runs after mutating, cannot modify	Enforce naming conventions, block privileged pods

```yaml
# MutatingWebhookConfiguration example
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: sidecar-injector
webhooks:
- name: inject.sidecar.example.com
  rules:
  - apiGroups: [""]
    apiVersions: ["v1"]
    resources: ["pods"]
    operations: ["CREATE"]
  clientConfig:
    service:
      namespace: istio-system
      name: istiod
      path: "/inject"
  admissionReviewVersions: ["v1"]
  sideEffects: None
  failurePolicy: Fail    # Fail-closed — safer than Ignore
```

Pod Security Admission (PSA)

PSA is the **built-in** pod security mechanism since Kubernetes 1.25 (replaces deprecated PodSecurityPolicy).

Three security levels:

Level	Restrictions
privileged	Unrestricted — only for trusted system workloads
baseline	Prevents known privilege escalation — allows most workloads
restricted	Maximum hardening — requires non-root, drops all caps, read-only root

Three modes:

Mode	Effect
`enforce`	Reject pods that violate the policy
`audit`	Allow pods but log violations to audit log
`warn`	Allow pods but return warnings to `kubectl`

Configuring Pod Security Admission

```yaml
# Apply PSA via namespace labels
apiVersion: v1
kind: Namespace
metadata:
  name: production
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

```
# Test what would be rejected before enforcing
kubectl label namespace staging \
  pod-security.kubernetes.io/warn=restricted

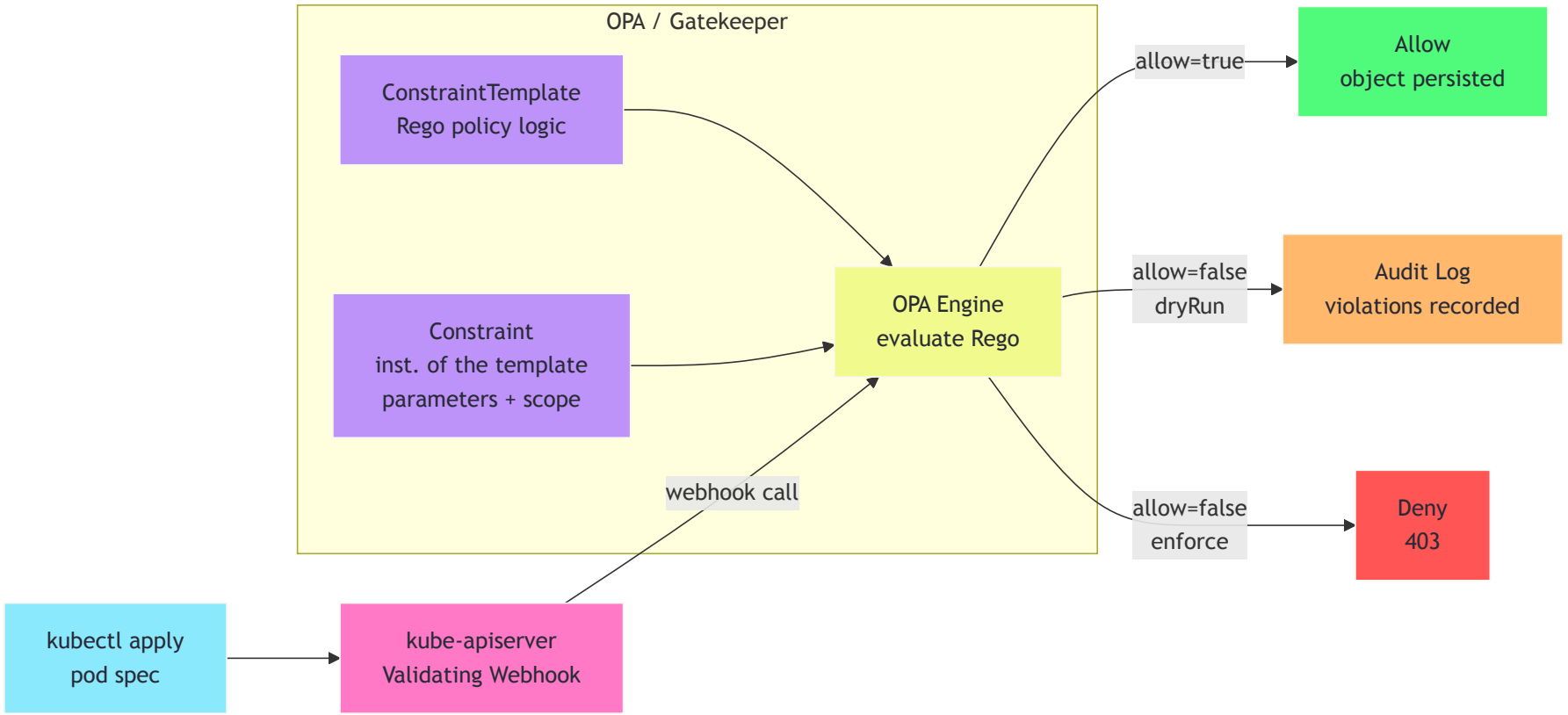
# Deploy a pod to see warnings
kubectl apply -f privileged-pod.yaml -n staging
# Warning: would violate PodSecurity "restricted:latest":
#   allowPrivilegeEscalation != false, runAsNonRoot != true
```

Pod that Passes the `restricted` Level

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    seccompProfile:
      type: RuntimeDefault      # applies default seccomp profile
  containers:
  - name: app
    image: ghcr.io/myorg/app:1.2.3
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]           # required by restricted level
    volumeMounts:
    - name: tmp
      mountPath: /tmp
  volumes:
  - name: tmp
    emptyDir: {}                # writeable scratch space
```

OPA / Gatekeeper

Open Policy Agent (OPA) is a general-purpose policy engine. **Gatekeeper** integrates OPA into Kubernetes as a validating admission webhook.



Gatekeeper — ConstraintTemplate and Constraint

```yaml
# ConstraintTemplate — defines the policy logic in Rego
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8srequiredlabels
spec:
  crd:
    spec:
      names:
        kind: K8sRequiredLabels
      validation:
        openAPIV3Schema:
          properties:
            labels:
              type: array
              items:
                type: string
  targets:
  - target: admission.k8s.gatekeeper.sh
    rego: |
      package k8srequiredlabels
      violation[{"msg": msg}] {
        provided := {label | input.review.object.metadata.labels[label]}
        required := {label | label := input.parameters.labels[_]}
        missing := required - provided
        count(missing) > 0
        msg := sprintf("Missing required labels: %v", [missing])
      }
```

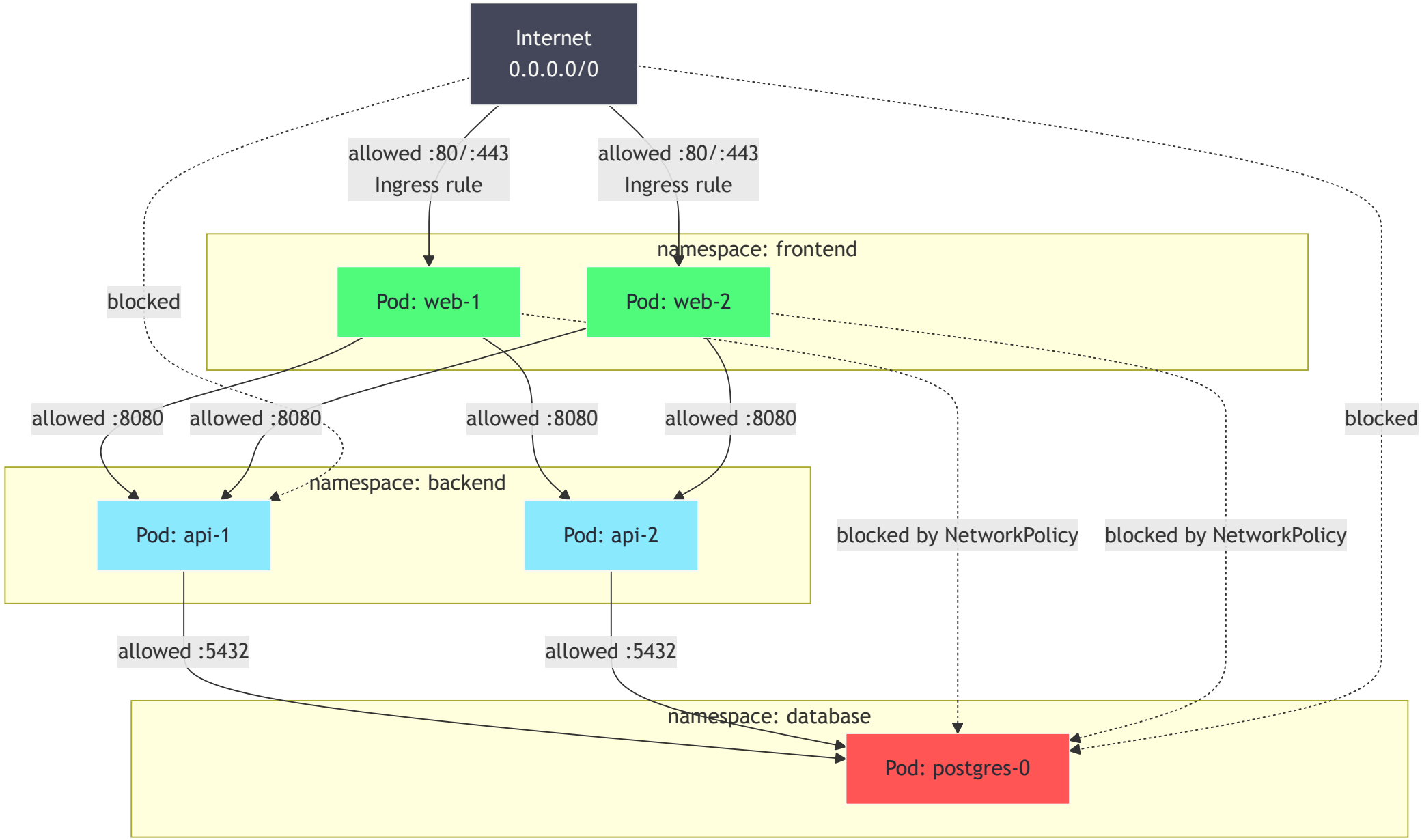
Gatekeeper — Instantiate a Constraint

```yaml
# Constraint — instance of the template with parameters
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: require-team-label
spec:
  match:
    kinds:
    - apiGroups: ["apps"]
      kinds: ["Deployment"]
    namespaces: ["production", "staging"]
  parameters:
    labels: ["team", "app", "version"]
```

```
# Test — deploy a Deployment without the required labels
kubectl apply -f unlabeled-deployment.yaml -n production
# Error: admission webhook "validation.gatekeeper.sh" denied
# the request: Missing required labels: {"team"}
```

Network Policies

By default, all pods in a Kubernetes cluster can communicate with all other pods on all ports. **NetworkPolicy** resources restrict that.


Internet
0.0.0.0/0
allowed :80/:443
Ingress rule
allowed :80/:443
Ingress rule
namespace: frontend
Pod: web-1
Pod: web-2
blocked
allowed :8080
allowed :8080
allowed :8080
allowed :8080
blocked
namespace: backend
Pod: api-1
Pod: api-2
blocked by NetworkPolicy
blocked by NetworkPolicy
allowed :5432
allowed :5432
namespace: database
Pod: postgres-0

Writing Network Policies

```yaml
# Default-deny all ingress in a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: backend
spec:
  podSelector: {}        # matches all pods
  policyTypes:
  - Ingress              # deny all ingress; egress still allowed

---
# Allow only frontend pods to reach backend on port 8080
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: frontend
      podSelector:
        matchLabels:
          app: web
    ports:
    - protocol: TCP
      port: 8080
```

Network Policy — Default Deny All

The gold standard: **deny all** at the namespace level, then explicitly allow what is needed.

```yaml
# 1. Deny all ingress and egress
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress

---
# 2. Allow DNS (required by all pods)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns
  namespace: production
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    ports:
```

Cluster Hardening

CIS Kubernetes Benchmark provides scored controls for hardening a cluster. Key areas:

```
# Run kube-bench to audit against CIS benchmark
kubectl apply -f https://raw.githubusercontent.com/aquasecurity/kube-bench/main/job.yaml
kubectl logs job/kube-bench

# Example failing control:
# [FAIL] 1.2.1 Ensure that the --anonymous-auth argument is set to false
# [FAIL] 1.2.6 Ensure that the --kubelet-certificate-authority argument is set
```

API Server Hardening Flags

```
# kube-apiserver flags (kubeadm: /etc/kubernetes/manifests/kube-apiserver.yaml)
spec:
  containers:
  - command:
    - kube-apiserver
    - --anonymous-auth=false              # require authentication always
    - --audit-log-path=/var/log/kube-audit.log  # audit logging
    - --audit-log-maxage=30
    - --audit-log-maxbackup=10
    - --audit-log-maxsize=100
    - --enable-admission-plugins=NodeRestriction,PodSecurity
    - --tls-min-version=VersionTLS13      # TLS 1.3 only
    - --authorization-mode=Node,RBAC     # no AlwaysAllow!
    - --encryption-provider-config=/etc/kubernetes/enc.yaml   # secrets at rest
```

Secrets Encryption at Rest

```yaml
# /etc/kubernetes/enc.yaml — EncryptionConfiguration
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
  - secrets
  - configmaps
  providers:
  - aescbc:
      keys:
      - name: key1
        secret: <base64-encoded-32-byte-key>   # openssl rand -base64 32
  - identity: {}    # fallback — reads unencrypted legacy secrets
```

```
# After enabling, re-encrypt existing secrets
kubectl get secrets -A -o json | kubectl replace -f -

# Verify a secret is encrypted in etcd
etcdctl get /registry/secrets/default/mysecret | xxd | head
# Should show encrypted binary, not readable YAML
```

Part 2 — Backup Strategies

Why Back Up a Kubernetes Cluster?

etcd holds all cluster state — if it's lost or corrupted, the cluster is gone.

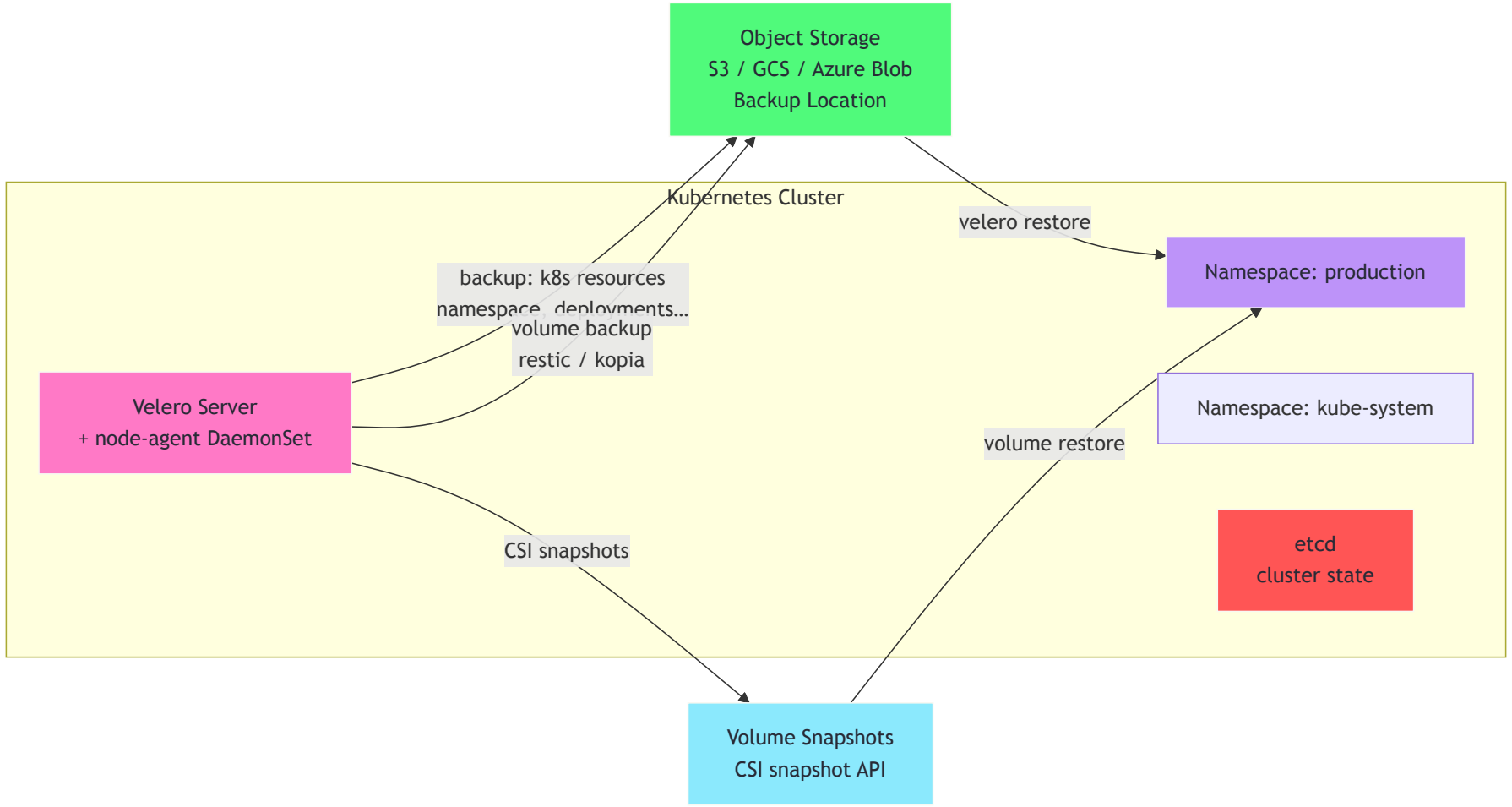
Risk	Scenario
etcd data loss	Disk failure on control plane nodes
Accidental deletion	`kubectl delete namespace production`
Cluster migration	Move to new cloud provider
Ransomware	Encrypted etcd
Upgrade failure	Can't roll back a broken upgrade
Multi-region DR	Restore in a different region

Two layers to back up:

1. **Cluster state** — all Kubernetes objects (Deployments, Services, ConfigMaps, Secrets, CRDs...) stored in etcd
2. **Persistent volumes** — application data stored in PVCs

Velero Architecture

Velero is the CNCF standard for Kubernetes backup and disaster recovery.



Velero — What It Backs Up

Resource type	How
Kubernetes objects	Serialised from etcd via the API server
PVC data	Via restic/kopia (file-level) or CSI volume snapshots
Cluster-scoped resources	Nodes, ClusterRoles, CRDs (optional)
Custom Resource Definitions	Backed up by default if in scope
Secrets	Backed up as-is — ensure object storage is encrypted

```
# Install Velero (AWS S3 example)
velero install \
  --provider aws \
  --plugins velero/velero-plugin-for-aws:v1.8 \
  --bucket my-velero-backups \
  --backup-location-config region=eu-central-1 \
  --snapshot-location-config region=eu-central-1 \
  --use-node-agent                # enables restic/kopia for PVC data
```

Velero — Backups and Schedules

```
# On-demand backup of a namespace
velero backup create production-backup \
  --include-namespaces production \
  --include-cluster-resources=true \
  --wait

# Scheduled backups (cron syntax)
velero schedule create daily-production \
  --schedule="0 2 * * *" \
  --include-namespaces production \
  --ttl 720h                    # keep for 30 days

# List backups
velero backup get

# Describe backup (check warnings)
velero backup describe production-backup --details

# Download backup logs
velero backup logs production-backup
```

Velero — Restore

```
# Restore a full backup
velero restore create --from-backup production-backup

# Restore a single namespace
velero restore create \
  --from-backup production-backup \
  --include-namespaces production

# Restore to a different namespace
velero restore create \
  --from-backup production-backup \
  --namespace-mappings production:production-dr

# Monitor restore progress
velero restore describe <restore-name> --details
kubectl get events -n production --field-selector reason=Restored

# Verify restored resources
kubectl get all -n production
kubectl get pvc -n production
```

etcd Backup — Direct Approach

For clusters where Velero isn't available, back up etcd directly.

```
# Snapshot etcd (run on control plane node)
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/healthcheck-client.crt \
  --key=/etc/kubernetes/pki/etcd/healthcheck-client.key

# Verify snapshot
ETCDCTL_API=3 etcdctl snapshot status /tmp/etcd-backup.db

# Copy off-cluster immediately
scp /tmp/etcd-backup.db backup@10.0.0.50:/backups/

# Restore (after cluster rebuild)
ETCDCTL_API=3 etcdctl snapshot restore /tmp/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored
```

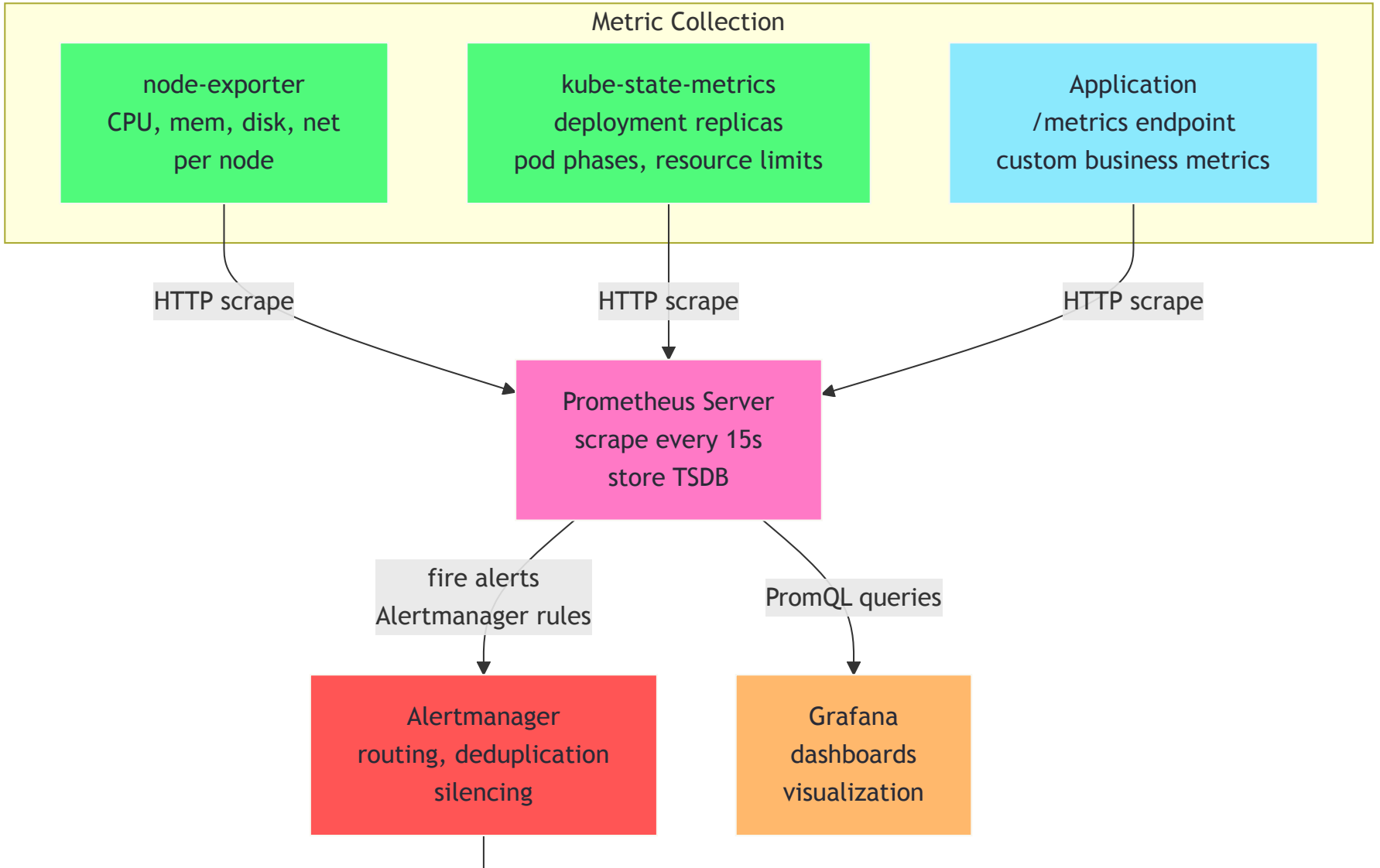
Part 3 — Monitoring

The Three Pillars of Observability

Pillar	What it tells you	Tooling
Metrics	Numeric time-series data — CPU, memory, request rate, error rate	Prometheus, Grafana
Logs	Structured or unstructured text events from processes	Loki, Elasticsearch, Fluentd
Traces	Distributed call chains across services	Jaeger, Tempo, OpenTelemetry

> Without observability, you are operating **blind**. When something breaks at 3am, metrics tell you what broke, logs tell you why, and traces tell you where.

Prometheus Architecture



Prometheus — How It Works

Prometheus uses a **pull model** — it scrapes `/metrics` HTTP endpoints at regular intervals.

```
# Example metrics exposed by an application
# HELP http_requests_total Total HTTP requests received
# TYPE http_requests_total counter
http_requests_total{method="GET", status="200"} 1027
http_requests_total{method="GET", status="404"} 12
http_requests_total{method="POST", status="500"} 3

# HELP request_duration_seconds HTTP request latency
# TYPE request_duration_seconds histogram
request_duration_seconds_bucket{le="0.01"} 850
request_duration_seconds_bucket{le="0.1"}  980
request_duration_seconds_bucket{le="1.0"}  1025
request_duration_seconds_sum 42.3
request_duration_seconds_count 1027
```

PromQL — Querying Metrics

PromQL is the query language for Prometheus. Used in Grafana dashboards and alerting rules.

```
# Current memory usage per pod (in MB)
container_memory_working_set_bytes{namespace="production"} / 1024 / 1024

# 5-minute HTTP error rate (as a ratio)
rate(http_requests_total{status=~"5.."}[5m])
  /
rate(http_requests_total[5m])

# CPU usage per container (as percentage of limit)
rate(container_cpu_usage_seconds_total[5m])
  /
on(pod, container) kube_pod_container_resource_limits{resource="cpu"}

# Pods not in Running state per namespace
count by (namespace) (kube_pod_status_phase{phase!="Running"} == 1)

# Alert: deployment replicas below desired
```

Alertmanager

Alertmanager receives alerts fired by Prometheus and routes them to the right people via the right channel.

```
# alertmanager.yml
route:
  group_by: ['alertname', 'namespace']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 12h
  receiver: 'slack-ops'
  routes:
  - match:
      severity: critical
    receiver: 'pagerduty-oncall'
  - match:
      team: backend
    receiver: 'slack-backend'

receivers:
- name: 'slack-ops'
  slack_configs:
  - api_url: 'https://hooks.slack.com/services/...'
    channel: '#ops-alerts'
    title: '{{ .GroupLabels.alertname }}'
    text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end }}'
```

Writing Alerting Rules

```yaml
# PrometheusRule resource (if using prometheus-operator)
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: production-alerts
  namespace: monitoring
spec:
  groups:
  - name: pods
    rules:
    - alert: PodCrashLooping
      expr: rate(kube_pod_container_status_restarts_total[15m]) * 60 * 15 > 0
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Pod {{ $labels.pod }} is crash-looping"
        description: "Pod {{ $labels.namespace }}/{{ $labels.pod }} has restarted more than once in the last 15 minutes."

    - alert: DeploymentReplicasMismatch
      expr: kube_deployment_spec_replicas != kube_deployment_status_replicas_available
      for: 10m
      labels:
        severity: critical
      annotations:
        summary: "Deployment {{ $labels.deployment }} has unavailable replicas"
```

kube-state-metrics and node-exporter

kube-state-metrics

Exposes the **state of Kubernetes objects** as metrics — what the API server knows about your cluster.

```
kube_deployment_replicas{deployment="web", namespace="production"} 3
kube_pod_status_phase{pod="web-abc123", phase="Running"} 1
kube_node_status_condition{condition="Ready", status="true"} 1
```

node-exporter

Exposes **host-level hardware and OS metrics** — runs as a DaemonSet.

```
node_cpu_seconds_total{cpu="0", mode="idle"} 12345.67
node_memory_MemAvailable_bytes 4294967296
node_disk_io_time_seconds_total{device="sda"} 42.5
node_filesystem_avail_bytes{mountpoint="/"} 107374182400
node_network_receive_bytes_total{device="eth0"} 987654321
```

Grafana — Dashboards

```
# Deploy Prometheus + Grafana stack via Helm
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --values monitoring-values.yaml

# Included dashboards (via Grafana provisioning):
# - Kubernetes / Cluster Overview
# - Kubernetes / Nodes
# - Kubernetes / Workloads
# - Kubernetes / Namespaces
# - Alertmanager / Overview
```

monitoring-values.yaml — Key Settings

```yaml
# monitoring-values.yaml
grafana:
  adminPassword: "changeme"
  ingress:
    enabled: true
    hosts: ["grafana.example.com"]
  persistence:
    enabled: true
    size: 10Gi

prometheus:
  prometheusSpec:
    retention: 15d
    storageSpec:
      volumeClaimTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi
    resources:
      requests:
        cpu: 500m
        memory: 1Gi
      limits:
        cpu: 2
        memory: 4Gi

alertmanager:
  alertmanagerSpec:
    storage:
      volumeClaimTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 5Gi
```

Incident Response in a Container Environment

When an alert fires, follow this procedure:

```
# 1. Identify the failing workload
kubectl get events -n production --sort-by='.lastTimestamp' | tail -20
kubectl get pods -n production | grep -v Running

# 2. Inspect the failing pod
kubectl describe pod <failing-pod> -n production   # look at Events section
kubectl logs <failing-pod> -n production --previous   # logs from crashed container

# 3. Check resource pressure
kubectl top nodes
kubectl top pods -n production --sort-by=memory
```

```
# 4. Check if the problem is node-level
kubectl describe node <affected-node>   # look at Conditions and Events

# 5. Isolate — cordon the node if needed
kubectl cordon <affected-node>
kubectl drain <affected-node> --ignore-daemonsets --delete-emptydir-data

# 6. Rollback if the issue was introduced by a deployment
kubectl rollout undo deployment/web -n production
```

Logging in Kubernetes

Container logs are ephemeral — when a pod is deleted, its logs are gone. Use a centralised log aggregation stack.



```
# Always write logs to stdout/stderr — do NOT write to files inside container
# Kubernetes captures stdout/stderr automatically

# Structured JSON logging is preferred (parseable by log aggregators)
{"level":"info","ts":"2026-04-21T10:00:00Z","msg":"request","path":"/api/users","latency_ms":12}

# Tail logs across multiple pods with a label selector
kubectl logs -l app=web -n production --tail=50 -f
```

Day 3 — Summary

Topic	Key Takeaway
RBAC	Least privilege — named resources, no wildcard verbs, dedicated ServiceAccounts
Admission Controllers	Mutating first, then validating; use `failurePolicy: Fail`
Pod Security Admission	Enforce `restricted` on all production namespaces
OPA/Gatekeeper	Policy as code — ConstraintTemplates + Constraints
Network Policies	Default-deny all; explicitly allow minimum required paths
Cluster hardening	Run kube-bench, disable anonymous auth, TLS 1.3, encrypt secrets at rest

Day 3 — Exercises

- **Exercise 1** — Configure RBAC, Pod Security Admission, and an OPA/Gatekeeper policy
- **Exercise 2** — Deploy Prometheus & Grafana, perform a Velero backup and namespace restore

See `exercises/day-3/` for full lab instructions.